

1. [10 Marks] Analyze the functional and operational requirements and propose a suitable software process model for the given scenarios. A. Online Banking Transaction System (Spiral Model)

- **Functional & Operational Requirements:** The system requires extreme data integrity, robust transaction processing, high-level security (encryption, access control), and absolute fault tolerance. Because a failure implies massive financial loss, risk management is the most critical operational requirement.
- **Technical Justification:** The **Spiral Model** is the most suitable because it is a risk-driven approach. An online banking system carries high security and financial risks; the spiral model explicitly addresses this by dedicating an entire phase in each iteration strictly to risk assessment, prototyping, and reduction before any development begins.
- **Structural Model Diagram Description:** Draw a spiral divided into four quadrants:
 1. *Objective Setting:* Define banking features and security constraints.
 2. *Risk Assessment & Reduction:* Evaluate fraud risks, create security prototypes.
 3. *Development & Validation:* Code and test the transaction module.
 4. *Planning:* Plan the next iteration.

B. University Examination Management System (Agile Model)

- **Functional & Operational Requirements:** The system needs user interfaces for students/faculty, grading logic, and report generation. The operational reality of universities is that rules (grading schemes, elective policies) change frequently, requiring high adaptability.
- **Technical Justification:** An **Agile/Incremental Development Model** is best. It allows the university to get core features (like student registration) running quickly. As examination rules change, the agile framework seamlessly accepts rapidly changing requirements and delivers new features in short, iterative releases.
- **Structural Model Diagram Description:** Draw an incremental flow: *Define outline requirements → Assign requirements to increments → Design architecture → Develop system increment → Validate increment → Integrate increment → Final System.*

2. [5 Marks] Principles of Software Project Management A common-sense approach to software project management relies on the following core principles:

- **Start on the right foot:** Work hard to understand the problem fully and set realistic, achievable objectives and customer expectations.
- **Maintain momentum:** Provide incentives to keep team turnover low. Emphasize quality in every task, and protect the team from unnecessary senior management interference.
- **Track progress:** Continuously track progress objectively by evaluating completed work products (models, source code, test cases) rather than relying on gut feelings.
- **Make smart decisions:** The project manager and the software team should always adhere to the principle of "keeping things simple."
- **Conduct a postmortem analysis:** Establish a consistent mechanism to extract and document "lessons learned" after every project to improve future processes.

3. [10 Marks] Data Flow Diagram for Hospital Management System

- **Definition:** A Data Flow Diagram (DFD) models the functional perspective of a system, showing how data is transformed from inputs to outputs.
- **Diagram Directions:**
 - **External Entities (Squares):** Patient, Doctor, Pharmacist.
 - **Processes (Rounded Rectangles/Circles):** Patient Registration, Appointment Booking, Medical Record Mgmt, Pharmacy Mgmt, Billing.

- **Data Stores (Open Rectangles): Patient DB, Appointments DB, Medical History DB, Inventory DB.**
- **Data Flows (Arrows):**
 1. **Patient** sends *Patient Details* → **Patient Registration** → writes to **Patient DB**.
 2. **Patient** sends *Request* → **Appointment Booking** (checks **Appointments DB**) → assigns slot with **Doctor**.
 3. **Doctor** sends *Diagnosis/Prescription* → **Medical Record Mgmt** → updates **Medical History DB**.
 4. **Medical Record Mgmt** sends *Prescription Data* → **Pharmacy Mgmt** (checks **Inventory DB**) → generates *Meds*.
 5. **Patient Registration, Appointment Booking, and Pharmacy Mgmt** send *Cost Data* → **Billing** → issues *Final Invoice* to **Patient**.

4. [5 Marks] Analyze factors and structure used to make a software project team To build an effective software team, managers must analyze several project factors and choose a complementary organizational structure.

- **Factors Considered:** The difficulty of the problem, the size of the program (LOC/FP), the team lifetime, the required quality/reliability, the rigidity of delivery dates, and the degree of communication required.
- **Team Structures (Constantine's Paradigms):**
 1. **Closed Paradigm:** A traditional hierarchy. Best for tight deadlines and routine projects similar to past work.
 2. **Random Paradigm:** A loose, flat structure that relies on individual initiative. Best for highly innovative or breakthrough projects.
 3. **Open Paradigm:** Blends hierarchical control with collaborative problem-solving. Best for complex problems requiring heavy communication.
 4. **Synchronous Paradigm:** Compartmentalizes the problem so members can work entirely independently with minimal communication.

5. [10 Marks] Numerical: Function Point (FP) Calculation *Given Data:* Productivity (OAP) = 20 FP/person-month; Labor cost = \$80,000/month; Count Total = 750; VAF = 1.17.

- **Step 1: Compute Total Estimated FP** Total FP = Count Total × VAF Total FP = 750 × 1.17 = **877.5 FP**
- **Step 2: Cost per Function Point** Cost per FP = Labor Cost per month / Productivity (OAP) Cost per FP = 80,000/20 = **\$4,000** per FP
- **Step 3: Overall Project Cost** Total Cost = Total FP × Cost per FP Total Cost = 877.5 × \$4,000 = **\$3,510,000**
- **Step 4: Estimated Effort (Requested as 'error' in prompt)** Estimated Effort = Total FP / Productivity (OAP) Estimated Effort = 877.5/20 = **43.875** person-months

6. [5 Marks] Application constraints for Non-Functional Requirements (Scalability) Non-functional requirements (NFRs) such as scalability, reliability, and security act as strict architectural constraints on a system.

- **Analysis:** NFRs restrict the developer's design choices. For example, if a system is given a strict **scalability** constraint (e.g., must support 10,000 concurrent users), the architecture is forcefully constrained to a distributed or Client-Server model rather than a centralized repository.
- **Metrics for Constraints:** To enforce these constraints properly, they must be measurable.
 - *Scalability/Speed:* Constrained by "Processed transactions/second" or "User response time".
 - *Reliability:* Constrained by "Mean time to failure" or "Availability percentage".
 - *Size:* Constrained by strict "M Bytes of RAM".

7. [10 Marks] State Machine Diagram

- **Definition:** A state machine diagram models the dynamic behaviour of a system in response to internal and external events. It is widely used for real-time and embedded systems.
- **Diagram Directions (Example: Microwave Oven):**
 - Draw rounded rectangles for the system states: **Waiting, Half_Power, Full_Power, Set_Time, Operation.**
 - Draw directed arrows between the states to represent transitions, and label them with the events/triggers:
 - **Waiting** Select Full Power **Full_Power**
 - **Waiting** Select Half Power **Half_Power**
 - **Full_Power** or **Half_Power** Input Time **Set_Time**
 - **Set_Time** Press Start **Operation**
 - **Operation** Timer Expires **Waiting**

8. [5 Marks] Describe W5HH principle Proposed by Barry Boehm, the W5HH principle is a universal project planning framework that helps managers understand the absolute essence of a project. It consists of 7 guiding questions:

1. **Why** is the system being developed? (*Establishes the business purpose*).
2. **What** will be done? (*Defines the task set*).
3. **When** will it be accomplished? (*Establishes the schedule*).
4. **Who** is responsible? (*Defines roles*).
5. **Where** are they organizationally located? (*Determines team location*).
6. **How** will the job be done technically and managerially? (*Defines process strategy*).
7. **How much** of each resource is needed? (*Leads to cost/effort estimation*).

9. [5 Marks] Principle of 4P's Risk Mitigation The Management Spectrum defines the 4 P's. Effectively managing them acts as the ultimate proactive risk mitigation strategy:

1. **People:** Mitigates *skill and morale risks*. Building a cohesive, properly trained, and motivated team ensures that staff turnover is minimized and technical challenges are handled effectively.
2. **Product:** Mitigates *scope creep and objective risks*. Strictly defining the product's scope, data, and functions upfront ensures that customer expectations remain realistic.
3. **Process:** Mitigates *technical and quality risks*. Choosing the right framework (e.g., Agile vs. Waterfall) ensures the development methodology inherently catches defects and handles changes.
4. **Project:** Mitigates *schedule and budget risks*. Continuous tracking, empirical cost estimation, and earned-value monitoring prevent the project from silently falling behind schedule.

10. [10 Marks] Classification diagram w.r.t to aggregation and generalization (Scenario: Petrol Bunk / Gas Station)

- **Generalization (Inheritance):** Shows an "is-a" relationship where specialized child classes inherit from a parent class.
 - *Diagram:* Draw a parent class box **Fuel**. Below it, draw child class boxes **Petrol** and **Diesel**. Connect them using a solid line ending in a **hollow triangle** pointing from the children upwards to the **Fuel** parent box.
- **Aggregation (Composition):** Shows a "has-a" or "part-of" relationship where a whole object is composed of smaller part objects.
 - *Diagram:* Draw a main container class box **Petrol_Bunk**. Draw smaller component boxes **Fuel_Pump**, **Storage_Tank**, and **Payment_Terminal**. Connect them to the main box using

a solid line. Place a **diamond symbol** on the line exactly where it touches the **Petrol_Bunk (Whole)** box.

11. [5 Marks] Factors causing software projects to get into trouble According to Pressman, software projects typically fall behind schedule or fail due to the following critical factors:

- **Unrealistic Deadlines:** Schedules arbitrarily established by managers outside the technical group.
 - **Scope Creep:** Changing customer requirements that are completely ignored in schedule adjustments.
 - **Poor Estimation:** An honest but inaccurate underestimation of the effort, cost, or resources required.
 - **Unmanaged Risks:** Predictable or unpredictable risks (technical, personnel) that were never planned for.
 - **Management Failure:** Failure by project management to recognize early warning signs that a project is falling behind, resulting in a lack of corrective action.
-

12. [5 Marks] Data-driven dictionaries

- **Definition:** A data dictionary is a central repository that maintains descriptions, logical forms, and structures of all the data entities used in a system design.
- **Role/Significance:**
 1. It ensures that every data entity (e.g., **Patient_ID**, **Engine_Type**) has a unique name and standardized definition, preventing fatal misinterpretations by different developers.
 2. It serves as the backbone for Computer-Aided Software Engineering (CASE) tools, allowing them to automatically cross-reference data, check for design consistency, and automatically generate comprehensive system documentation.